

UPPSALA
UNIVERSITET

BACHELOR THESIS SPECIFICATION

Evaluating Stainless on SV-COMP

Student:
Hadi Saleh
hadi.saleh.9490@student.uu.se
Signature

Supervisor:
Eva Darulova
eva.darulova@it.uu.se
Signature

Subject Reviewer:
Name
Email
Signature

Department of Information Technology

February 28, 2026

Contents

1 Title **3**

2 Abstract **3**

3 Background **3**

4 Description of Tasks **4**

5 Methods **5**

6 Relevant Courses **6**

7 Delimitations **6**

8 Time Plan **6**

1 Title

Evaluating Stainless on SV-COMP Benchmarks

2 Abstract

Formal verification aims to mathematically prove that software satisfies desired properties. This project evaluates Stainless[1] using benchmarks from SV-COMP (Competition on Software Verification)[2], a widely used benchmark collection for comparing verification tools. Since SV-COMP benchmarks are mostly written in the languages C and Java, the project first adapts a selected subset into equivalent Scala programs which become suitable for Stainless.

The evaluation then measures how many benchmarks Stainless can verify within given resource limits, verification time and scalability, and common reasons for verification failures or timeouts.

The outcome will be a structured assessment of Stainless on a standardized workload and recommendations on which benchmark patterns and properties are tractable.

3 Background

This thesis is conducted within the context of formal software verification, a field concerned with mathematically proving that programs behave correctly with respect to a given specification.

Stainless is a deductive verification framework developed at EPFL for the programming language Scala[3]. It supports automated verification of properties such as functional correctness, termination, and absence of runtime errors through the use of SMT solvers[4]. Stainless has previously been applied in both academic research and industrial case studies, demonstrating its potential as a practical verification tool.

However, its performance on large, standardized benchmark suites has not

been systematically evaluated, which limits our understanding of its scalability and general applicability.

SV-COMP is an annual international competition on software verification that maintains a large, curated benchmark suite covering a wide range of verification tasks, including safety properties, heap memory correctness, and concurrency. The benchmarks are widely used by the research community as a common basis for comparing verification tools in a reproducible manner.

Since SV-COMP benchmarks are primarily written in C or Java, they cannot be processed directly by Stainless and must first be adapted into equivalent Scala programs.

The goal of this project is to establish a clearer picture of where Stainless stands relative to standardized verification workloads, and to identify the kinds of programs and properties it can handle well versus where it struggles. SV-COMP benchmarks have not previously been adapted for use with Stainless, meaning the translation and annotation work carried out in this thesis is an original contribution.

4 Description of Tasks

The core problem this project addresses is the lack of systematic evaluation of Stainless on standardized verification benchmarks. Since SV-COMP benchmarks are written in C or Java, they cannot be directly used with Stainless, thus making the adaptation process a central and intellectually interesting challenge.

Careful analysis is also required when translating programs between languages while preserving their formal semantics. Additionally, determining what specifications and annotations are needed for Stainless to attempt verification demands a solid understanding of both the tool and the benchmarks themselves.

The work consists of the following main tasks:

1. **Benchmark selection.** Survey the SV-COMP benchmark categories and identify a reasonable subset which is suitable for translation to

Scala. However, priority will be given to Java benchmarks, as the Java to Scala conversion is a lot more straightforward than C to Scala, though an attempt of a limited number of C benchmarks may be explored for contrast.

2. **Benchmark translation.** Translate the selected benchmarks into equivalent Scala programs that Stainless can process, and adding necessary annotations (e.g., preconditions, postconditions, invariants) where required.

3. **Evaluation.** Run Stainless on the translated benchmarks and record verification outcomes (verified, failed, timeout, unknown) along with the runtime.

4. **Analysis.** Analyse the results to identify patterns in what Stainless handles well and where it struggles. Document these results and the types of programs or properties that cause difficulties.

5. **Report writing.** Write the thesis report documenting methodology, results, and conclusions continously along the other tasks.

5 Methods

The project follows an empirical evaluation methodology, where Stainless is treated as the system under study and its verification outcomes on adapted SV-COMP benchmarks serve as the primary data. The main tools involved are Stainless and the SV-COMP benchmark repository[5]. Benchmarks will be translated into Scala manually.

Results will be logged systematically, recording for each benchmark the verification outcome (verified, failed, timeout, or unknown) and the time taken. Both quantitative measures, such as the percentage of benchmarks verified within resource limits, and qualitative observations, such as recurring causes of failure, will be used to draw conclusions.

Relevant literature includes foundational work on deductive and SMT-based program verification, prior publications related to Stainless, and previous SV-COMP competition reports. Additional literature will be reviewed as

needed to put Stainless's results in context and compare them with other verification tools that have participated in SV-COMP. All findings will be documented in the thesis report, which will cover how benchmarks were selected and translated, how the experiments were set up, and have an analysis of the results.

6 Relevant Courses

- 1DL311 Semantics of Programming Languages
- 1DL201 Programkonstruktion och datastrukturer
- 1MA027 Logic and Proof Techniques I
- 1DL221 Imperative and OOP Methodology

7 Delimitations

To keep the project feasible within the thesis time frame, some delimitations will apply.

1. This project will mainly focus on the Java benchmarks from SV-COMP, since they are structurally closer to Scala and therefore easier to adapt to Stainless. If time permits, a limited number of C benchmarks may also be explored to evaluate how well they can be translated and verified.
2. SV-COMP contains a very large number of benchmarks, and it is not feasible to replicate or evaluate the full competition setup. Therefore, a logical and motivated subset of benchmarks will be selected. The selection will be based on factors such as relevance to the benchmarks, Stainless, feasibility of translation, and time constraints.

8 Time Plan

This project is expected to span approximately 10 weeks. Below is the proposed preliminary timeline:

- Weeks 12–13: Study Stainless and its annotation system, survey SV-COMP benchmark categories, and select a representative subset of benchmarks to translate.
- Weeks 13–15: Translate and annotate the selected Java benchmarks into Scala programs suitable for Stainless.
- Weeks 15–16: Set up the environment and run initial experiments to verify the translation process works as intended.
- Weeks 16–19: Run the full evaluation, systematically logging verification outcomes and runtimes across all translated benchmarks.
- Weeks 19–21: Analyze the results, identify patterns in verification successes and failures, and review relevant literature to contextualize findings.
- Weeks 12–21: Write the report continuously alongside the other tasks.
- Weeks 21–22: Finish writing the report, incorporate feedback from the supervisor, and prepare the final presentation.

Regular weekly meetings with the supervisor will take place throughout the project to discuss progress and address any challenges that arise.

References

[1] EPFL LARA. (2025, Sep.) Stainless and scala. EPFL. [Online]. Available: https://epfl-lara.github.io/stainless/intro.html

[2] D. Beyer. (2026) Competition on software verification (sv-comp). SoSy-Lab. [Online]. Available: https://sv-comp.sosy-lab.org/

[3] Scala Center and Lightbend, Inc. (2026) The scala programming language. Scala Center. [Online]. Available: https://www.scala-lang.org/

[4] C. Barrett, P. Fontaine, and C. Tinelli. (2016) The satisfiability modulo theories library (smt-lib). The SMT-LIB Initiative. [Online]. Available: https://smt-lib.org/

[5] SV-COMP Software Verification Benchmarks, "Sv-benchmarks," https://gitlab.com/sosy-lab/benchmarking/sv-benchmarks, 2026.

Signature page

This document has been electronically signed using eduSign.

eduSign

Electronically signed by
Eva Darulova Bruggisser
eduSign

Date and time of signature
2026-02-28 06:39 GMT

Authenticated by
Uppsala University

Electronically signed by
Hadi Saleh
eduSign

Date and time of signature
2026-02-28 11:46 GMT

Authenticated by
Uppsala universitet